Multicore OS

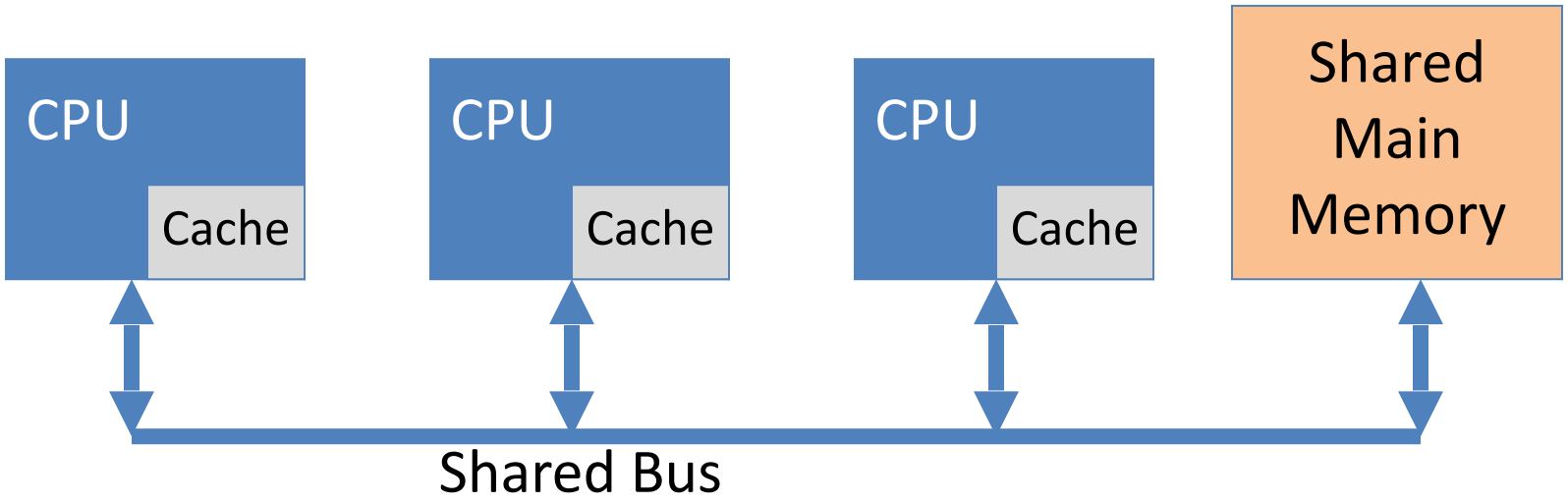
CS 202: Advanced Operating Systems

Motivation

- Multicores are the norm today
- Architectures become heterogeneous
  - Different general-purpose cores
  - Accelerators and specialized processors
  - Hard to share an OS with such differences
- Interconnects matter: within cores and across cores
- Sharing within the OS is becoming a problem
  - Cache-coherence protocol limits *scalability*
  - Prevents effective use of heterogeneous cores

Bus-based Shared Memory

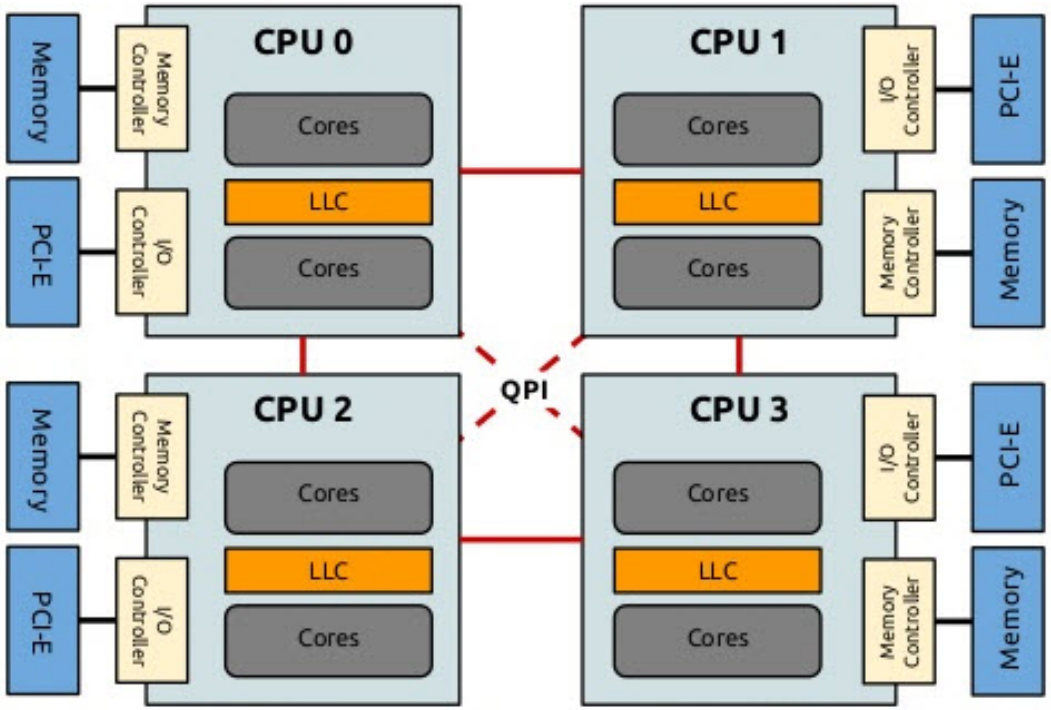
- Multiprocessor and multi-core systems
  - Bus bandwidth limits # of CPUs
  - Could be multiple memory elements (e.g., NUMA)
- Symmetric multiprocessor (SMP)
  - Uniform Memory Access
    - Access to all memory occurs at the same speed for all processors



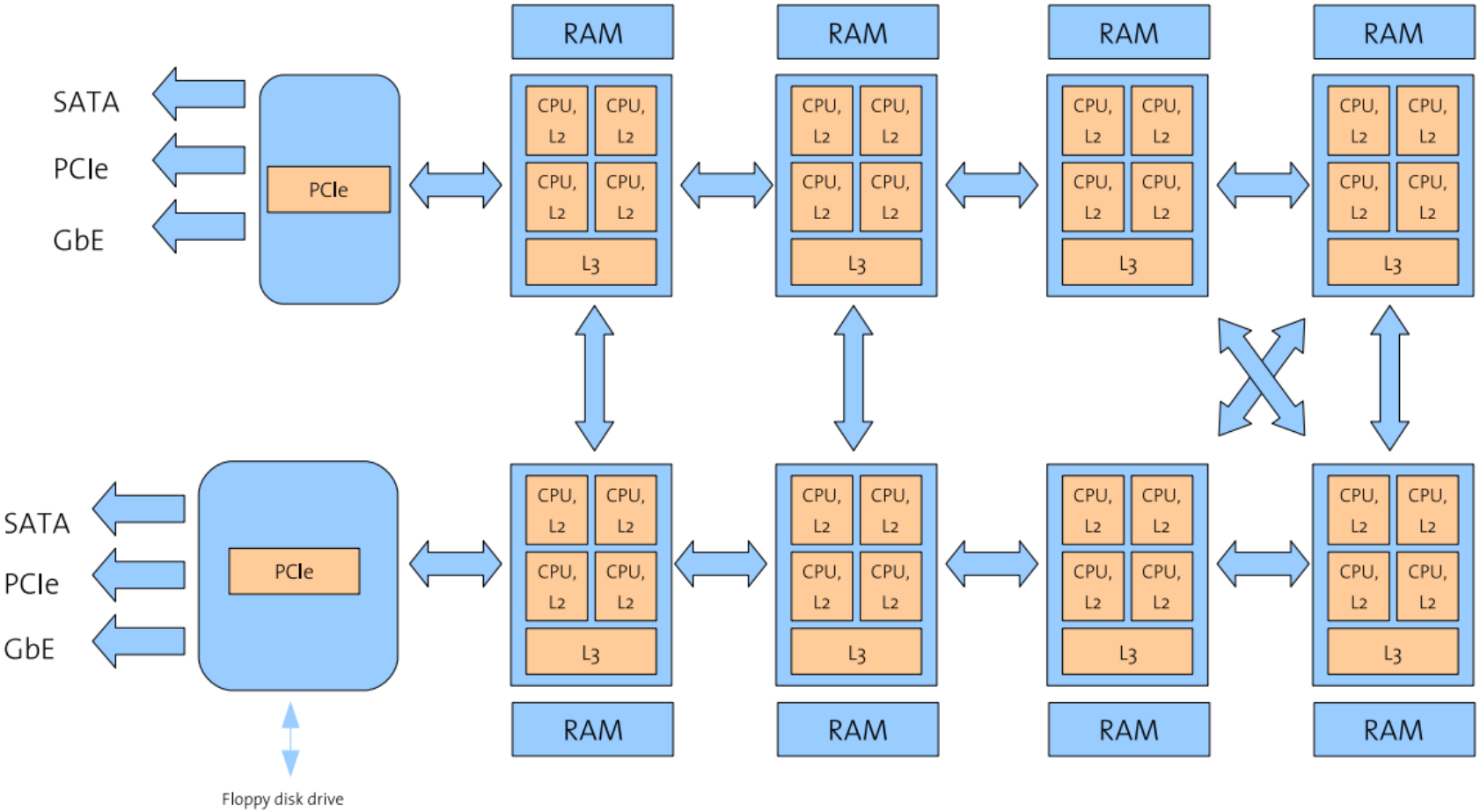
Non-Uniform Memory Access (NUMA)

- Access to some parts of memory is faster for some processors than other parts of memory
- Provides high-local bandwidth and reduces bus contention

CPU architecture (Intel Sandy Bridge)

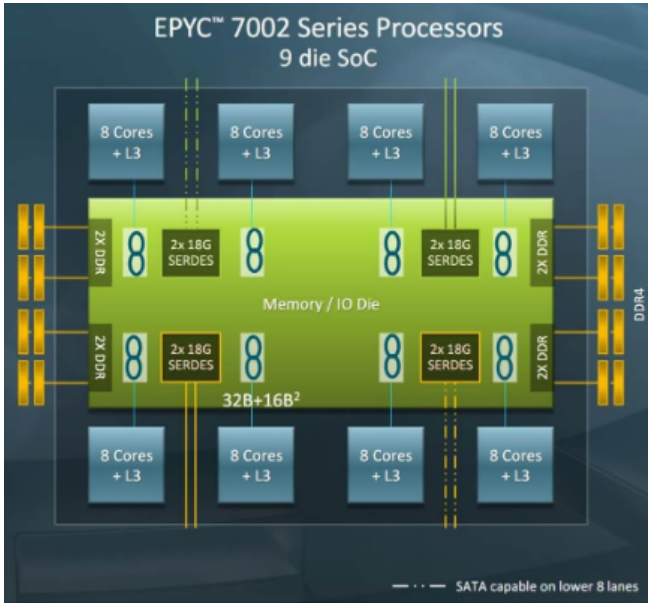


Interconnects

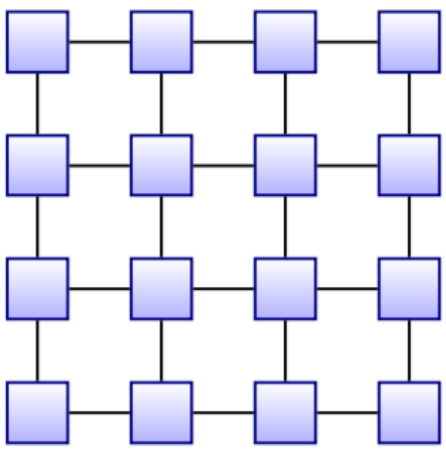


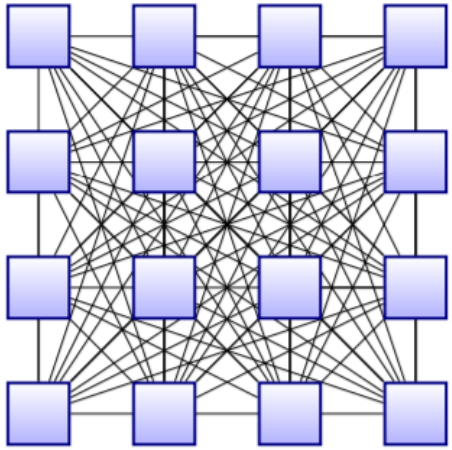
8×4-core AMD system

Interconnects are evolving



AMD EPYC 64 cores





"On-chip networks are more efficient in terms of power-consumption and area"*

* Evgeny Bolotin et al. "Cost Considerations in Network on Chip". In: Integration-The VLSI Journal, special issue on Network on Chip, Volume 38, Issue 38 (2004), pp. 105–128.

Scalability

- ***Application speedup/Number of cores*** ratio
- Ideally, but rarely, 100%
  - e.g., 100x speed up with 100 cores!
- Typically lesser due to
  - Inherently sequential part(s) of the application
  - Other bottlenecks
    - Obtaining locks on shared variables, …
    - Unnecessary sharing

Amdahl's Law

- Theoretical speedup gain (upper bound) by the enhancement of a particular part of the system
- Definition

$$SPEEDUP = \frac{1}{(1-p) + \frac{p}{s}}$$

- p: proportion of the enhanced part
- s: speedup of the enhanced part

Amdahl's Law

- Theoretical speedup gain (upper bound) by the enhancement of a particular part of the system
- Definition

$$SPEEDUP = \frac{1}{(1-p) + \frac{p}{s}}$$

- p: proportion of the enhanced part
- s: speedup of the enhanced part

- In multi-core systems with parallelism:
  - *p*: the proportion of the parallel part
  - *(1-p)*: the proportion of the serial part
  - *s*: the number of CPU cores for parallelism
- Insight: Reduce sequential parts, increase parallel parts!

Amdahl's Law Example

$$SPEEDUP = \frac{1}{(1-p) + \frac{p}{s}}$$

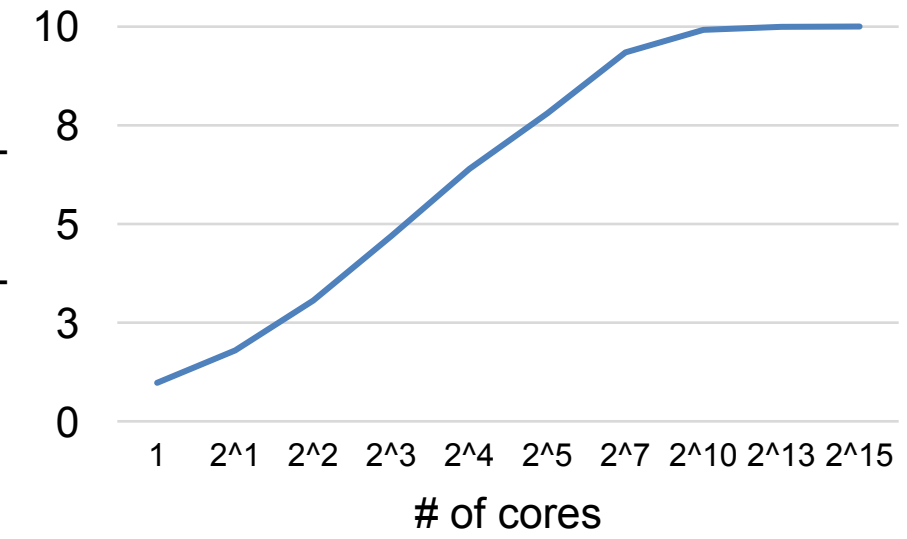
- Suppose we have a program that is 90% parallel and 10% serial Assume ideal parallelism in the parallel part.
  - p = 0.9, s = # of cores
  - 1 core: $\frac{1}{(1-0.9) + \frac{0.9}{1}} = 1$
  - 2 cores: $\frac{1}{(1-0.9) + \frac{0.9}{2}} = 1.82$
  - 4 cores: $\frac{1}{(1-0.9) + \frac{0.9}{4}} = 3.08$
  - 8 cores: $\frac{1}{(1-0.9) + \frac{0.9}{8}} = 4.71$

Amdahl's Law Example

$$SPEEDUP = \frac{1}{(1-p) + \frac{p}{s}}$$

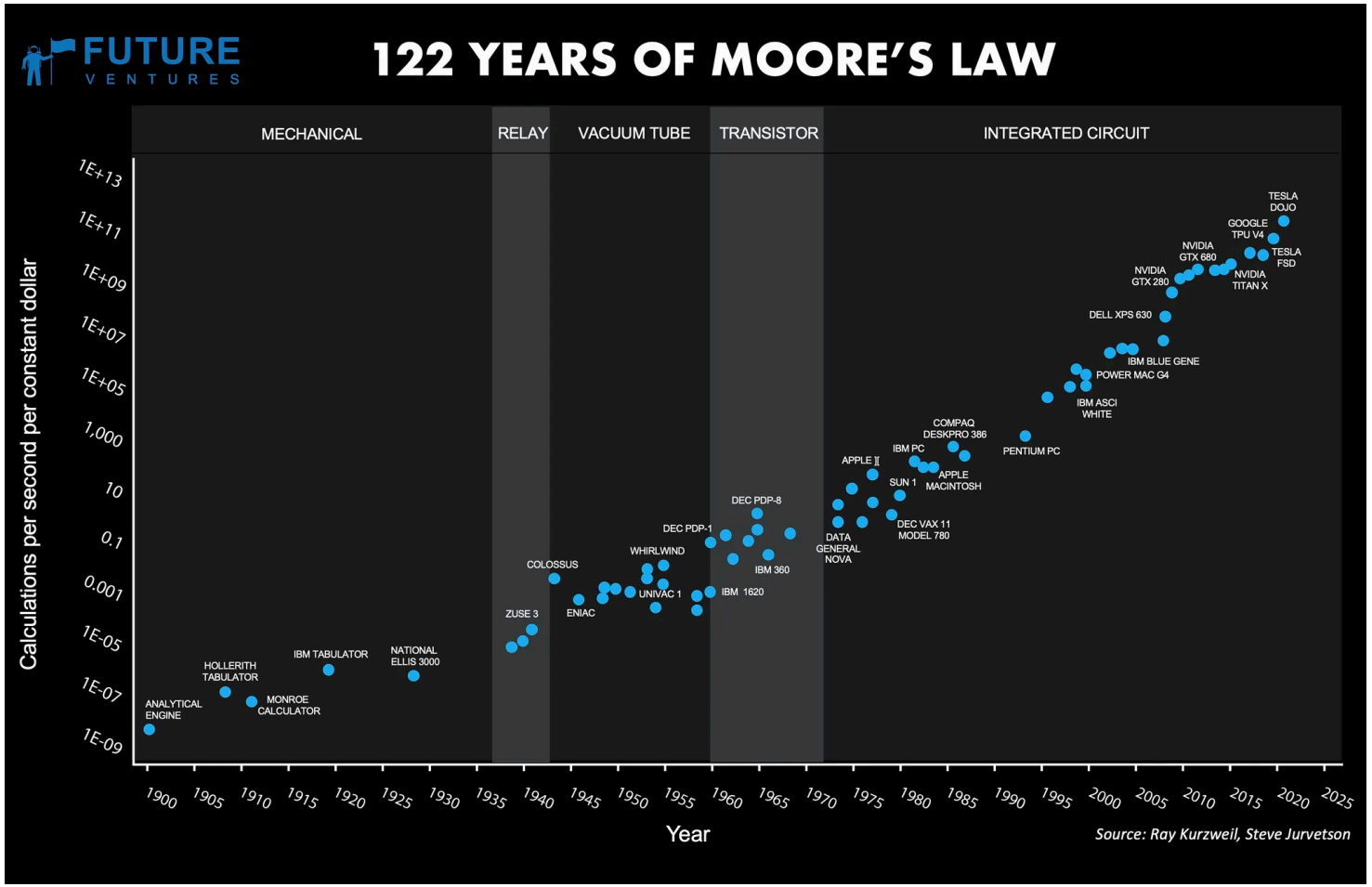
- Suppose we have a program that is 90% parallel and 10% serial Assume ideal parallelism in the parallel part.
  - p = 0.9, s = # of cores
  - 1 core: $\frac{1}{(1-0.9) + \frac{0.9}{1}} = 1$
  - 2 cores: $\frac{1}{(1-0.9) + \frac{0.9}{2}} = 1.82$
  - 4 cores: $\frac{1}{(1-0.9) + \frac{0.9}{4}} = 3.08$
  - 8 cores: $\frac{1}{(1-0.9) + \frac{0.9}{8}} = 4.71$

- What happens as s →∞?



- Speedup approaches 1/(1-p)
- Reduce serial portions
- Improve serial portions

Moore's Law



(pic from evobsession.com)

Are OSes ready for scalability?

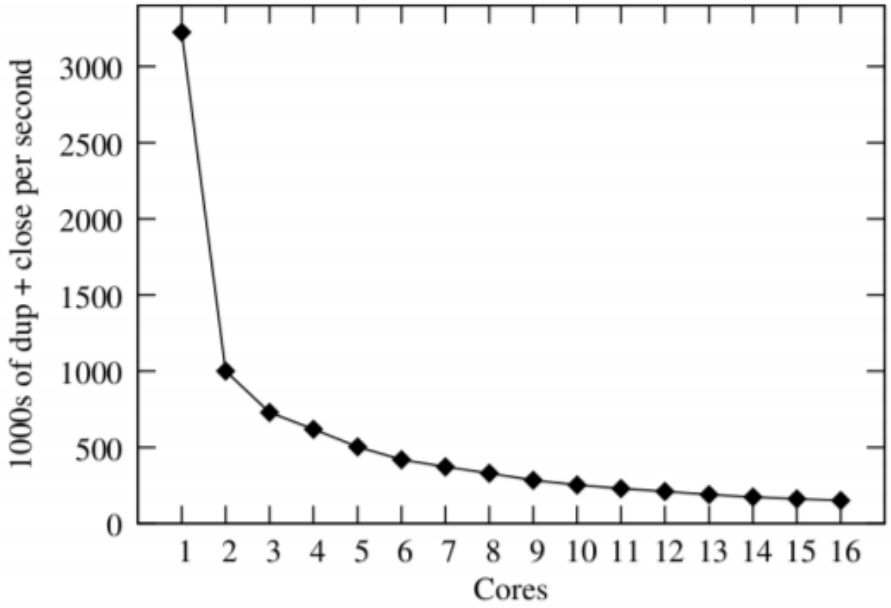
- In-kernel locking: *n* threads on *n* cores execute the following:
  - `dup()`: allocate a new file descriptor
  - Contention on the process's open file table protected by a lock

```
1 f = open("filename");
2
3 while (true) {
4  f2 = dup(f);
5  close (f2);
6 }
```

Are OSes ready for scalability?

- In-kernel locking: *n* threads on *n* cores execute the following:
  - `dup()`: allocate a new file descriptor
  - Contention on the process's open file table protected by a lock

```
1 f = open("filename");
2
3 while (true) {
4   f2 = dup(f);
5   close(f2);
6 }
```

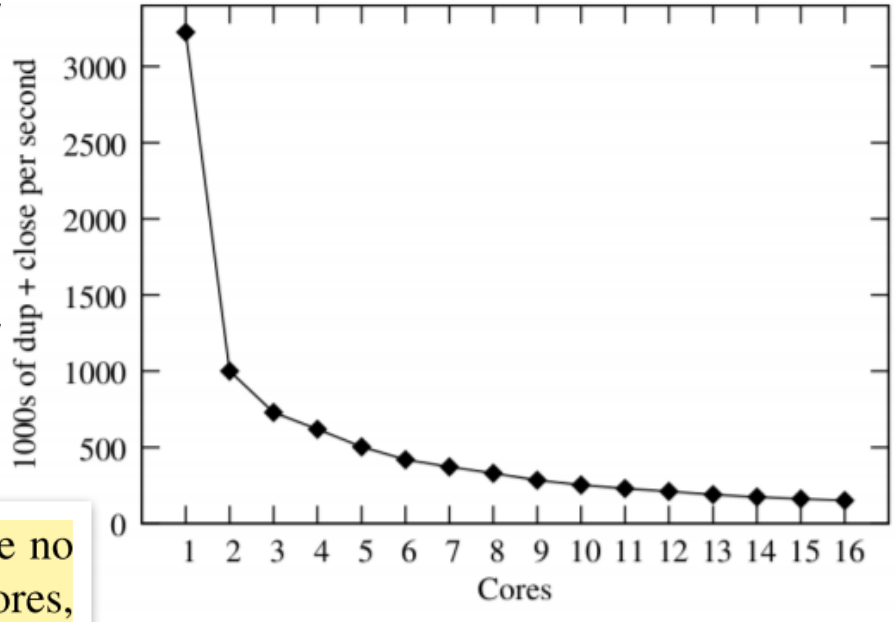


Are OSes ready for scalability?

- In-kernel locking: *n* threads on *n* cores execute the following:
  - `dup()`: allocate a new file descriptor
  - Contention on the process's open file table protected by a lock

```
1 f = open("filename");
2
3 while (true) {
4   f2 = dup(f);
5   close(f2);
6 }
```

Decreasing performance with increasing number of cores in Linux*



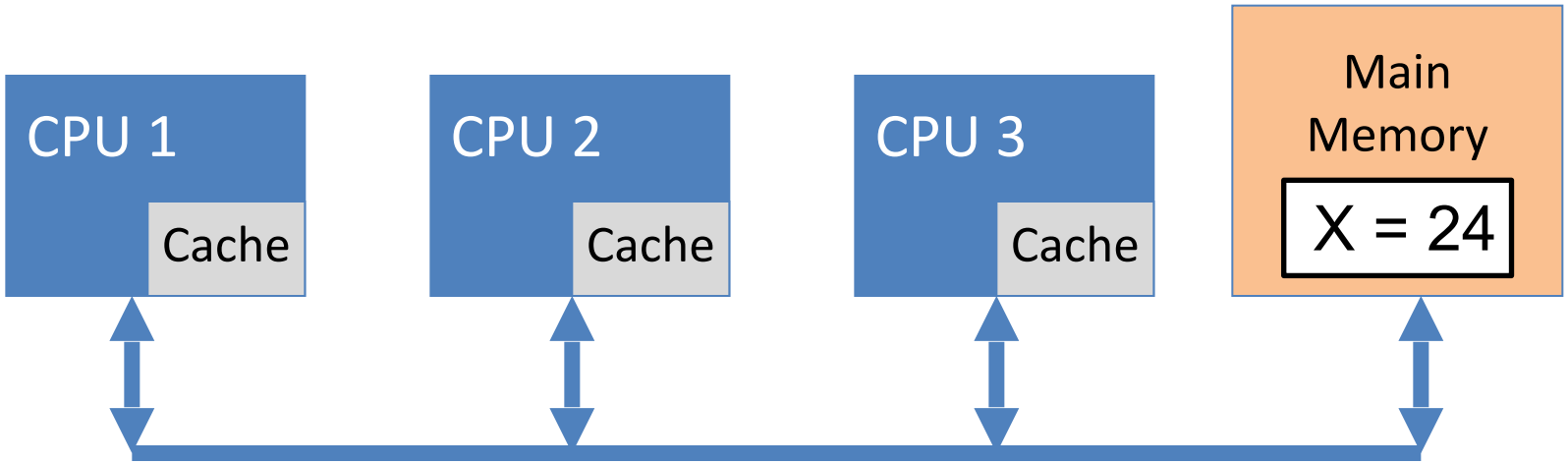
ing the process's open files. With one core there are no cache misses, and the benchmark is fast; with two cores, the cache coherence protocol forces a few cache misses per iteration to exchange the lock and table data. More

* Boyd-Wickizer, Silas, Haibo Chen, Rong Chen, Yandong Mao, M. Frans Kaashoek, Robert Tappan Morris, Aleksey Pesterev et al. "Corey: An Operating System for Many Cores." In OSDI, vol. 8, pp. 43-57. 2008.

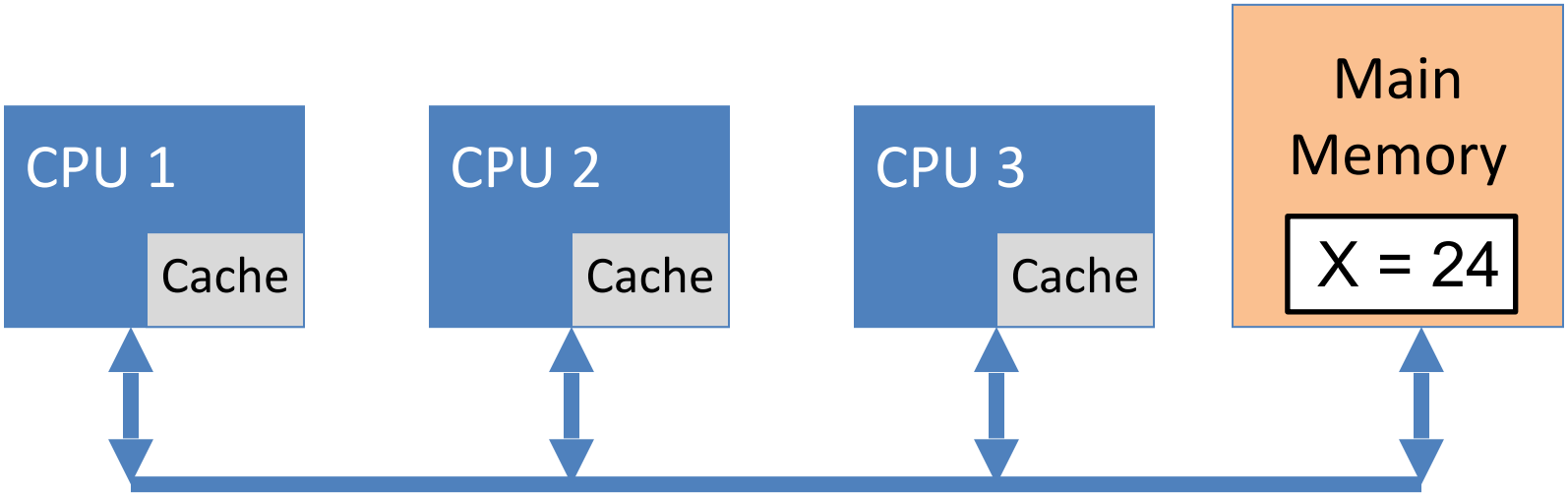
Background: Cache Coherence

- Cache coherence: the uniformity of shared resource data that ends up stored in multiple local caches

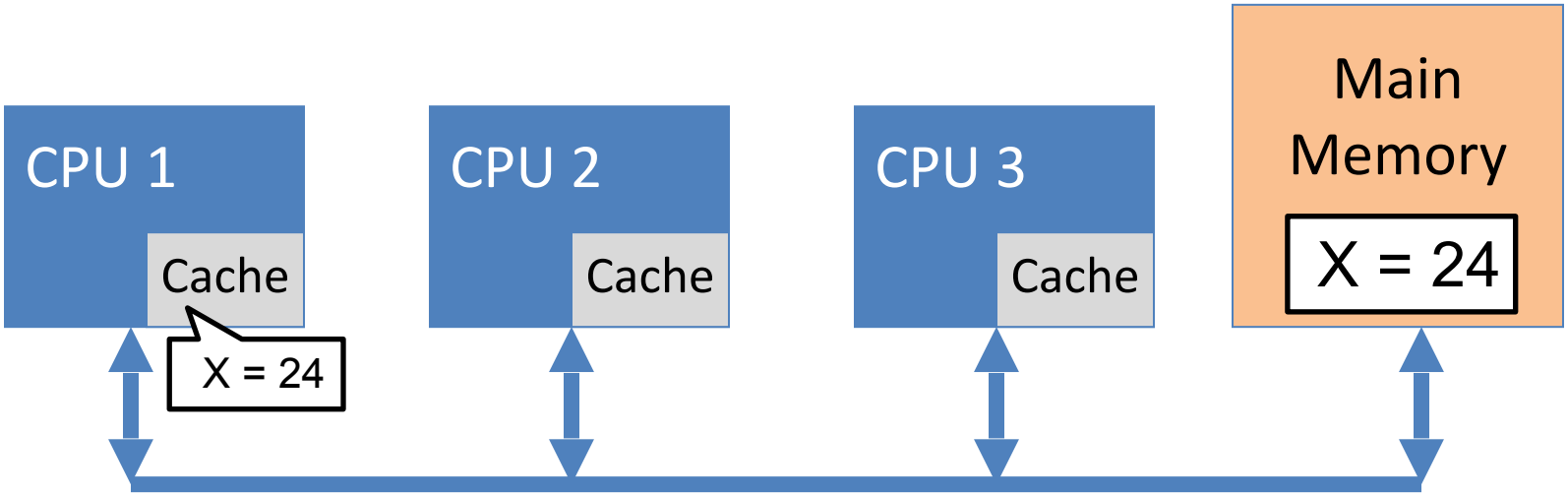
- Example: for simplicity, we'll assume
  - Just a single-level cache per CPU
  - Single memory element on a bus
  - X is a shared variable



Example

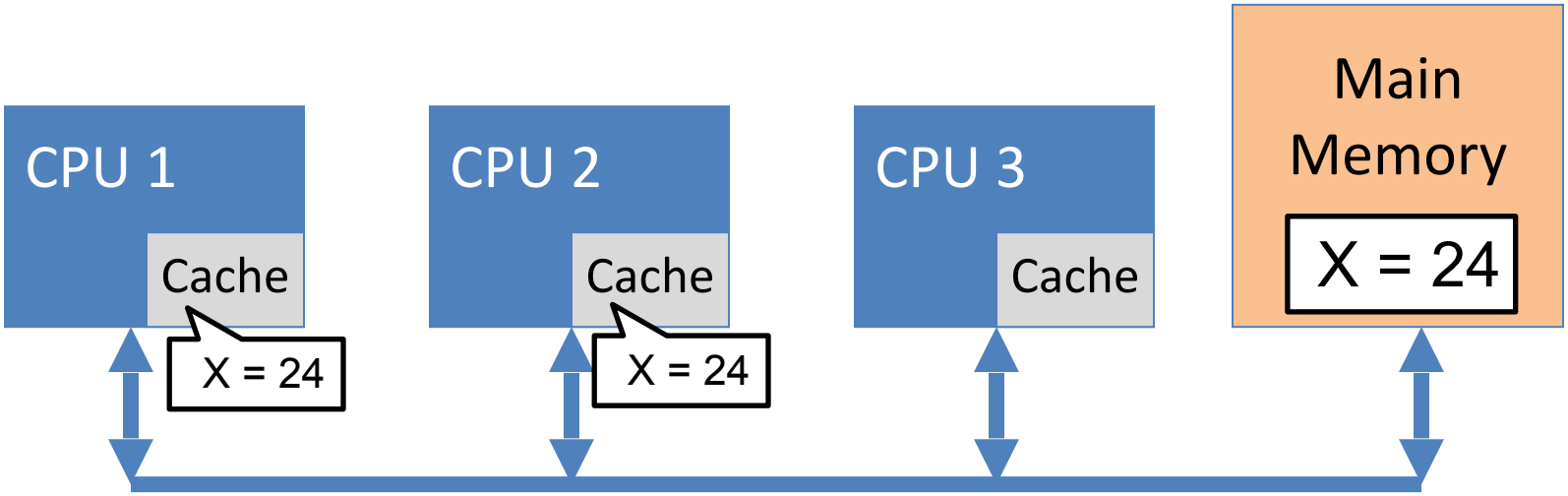


Example



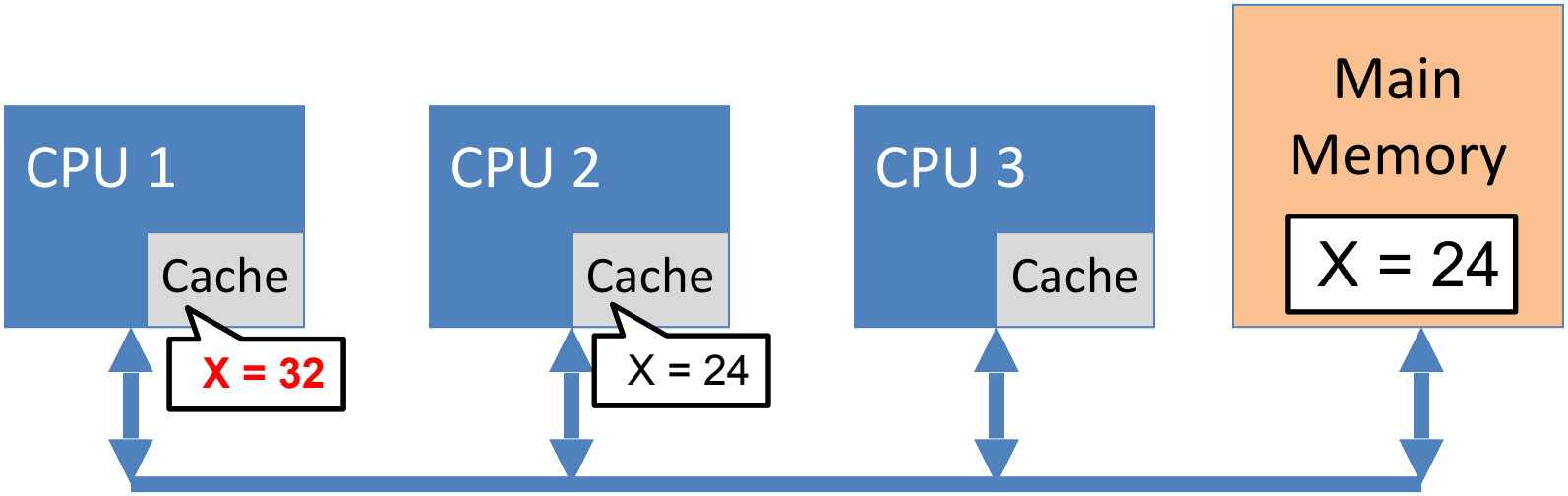
- CPU 1 reads X: Obtains 24 from memory and caches it

Example



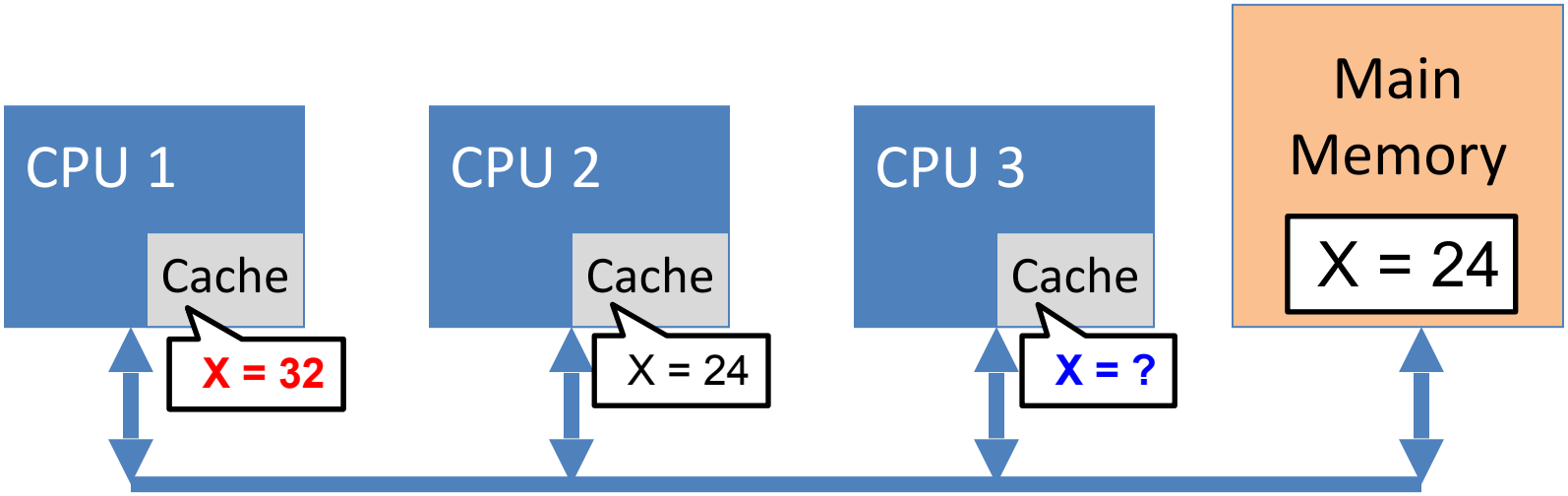
- CPU 1 reads X: Obtains 24 from memory and caches it
- CPU 2 reads X: Obtains 24 from memory and caches it

Example



- CPU 1 reads X: Obtains 24 from memory and caches it
- CPU 2 reads X: Obtains 24 from memory and caches it
- CPU 1 writes 32 to X: Its locally cached copy is updated

Example



- CPU 1 reads X: Obtains 24 from memory and caches it
- CPU 2 reads X: Obtains 24 from memory and caches it
- CPU 1 writes 32 to X: Its locally cached copy is updated
- CPU 3 reads X: What value should it get?
  Memory and CPU 2 think it is 24
  CPU 1 thinks it is 32

What Does Coherency Mean?

- Coherence defines the behavior of reads and writes to a single address location

What Does Coherency Mean?

- Coherence defines the behavior of reads and writes to a single address location

- Informally: Any read must return the most recent write
  - Too strict and very difficult to implement

What Does Coherency Mean?

- Coherence defines the behavior of reads and writes to a single address location

- Informally: Any read must return the most recent write
  - Too strict and very difficult to implement

- Better:
  1) Any write must eventually be seen by a read
  - If P1 writes X and P2 reads it, P1's write will be seen by P2 if the read and write are sufficiently far apart

What Does Coherency Mean?

- Coherence defines the behavior of reads and writes to a single address location

- Informally: Any read must return the most recent write
  - Too strict and very difficult to implement

- Better:

  1) Any write must eventually be seen by a read
     - If P1 writes X and P2 reads it, P1's write will be seen by P2 if the read and write are sufficiently far apart

  2) All writes are seen in proper order ("serialization")
     - Writes to a single location are serialized: seen in one order
     - Latest write will be seen

Cache Coherence

- Make every CPU know who has a copy of its cached data

Cache Coherence

- Make every CPU know who has a copy of its cached data

1. Snooping solution

Cache Coherence

- Make every CPU know who has a copy of its cached data

1. Snooping solution
   - Send all requests for data to all CPUs
   - Each CPU snoops memory bus to see and respond if it has a cached copy
     - Looks for read/write activities
   - Requires broadcast, since caching information is at processors
     - Works well with bus (natural broadcast medium)
     - Dominates for small scale machines (most of the market)

Cache Coherence

- Make every CPU know who has a copy of its cached data

1. Snooping solution
   - Send all requests for data to all CPUs
   - Each CPU snoops memory bus to see and respond if it has a cached copy
     - Looks for read/write activities
   - Requires broadcast, since caching information is at processors
     - Works well with bus (natural broadcast medium)
     - Dominates for small scale machines (most of the market)

2. Directory-based coherence schemes
   - One centralized place to keep track of which caches have copies of shared memory blocks
   - Distributed memory (NUMA) → distributed directory (avoids bottlenecks)
   - Send point-to-point requests to processors
   - Scales better than Snoop

Basic Snooping Protocols

1. Write Invalidate Protocol
   - Write to shared data: an ***invalidate*** is sent to all caches which snoop and invalidate any copies
   - Any shared read in other CPUs will now miss in cache and **re-fetch new data from memory**
2. Write Update Protocol
   - Write to shared data: broadcast on bus, processors snoop, and update copies (high bus traffic)

Basic Snooping Protocols

1. Write Invalidate Protocol
   - Write to shared data: an ***invalidate*** is sent to all caches which snoop and invalidate any copies
   - Any shared read in other CPUs will now miss in cache and **re-fetch new data from memory**
2. Write Update Protocol
   - Write to shared data: broadcast on bus, processors snoop, and update copies (high bus traffic)

- Bus bandwidth is a precious resource
  - Contention/cache interrogation can lead to >10x performance drop
  - Therefore, invalidate protocols used in most commercial SMPs
- In both schemes, writes are serialized by bus arbitration - only one CPU can use the bus at any one time

Revisit the spinlock implementation

- Test-And-Set (TAS) Lock

```
acquire(L):
        while(test_and_set(L));
release(L):
        L=0;
```

Test and Set: write 1 to the memory location and return its old value atomically

- It works, but...

Revisit the spinlock implementation

- Test-And-Set (TAS) Lock

```
acquire(L):
        while(test_and_set(L));
release(L):
        L=0;
```

Test and Set: write 1 to the memory location and return its old value atomically

- It works, but...
- Contention
  - Suppose many CPUs are waiting to get the lock
  - Test-and-set is a write operation
    - Has to go to memory
    - **A lot of cache coherence traffic** (write invalidate)
    - Unnecessary, unless the lock has been released

Better implementations (1)

- Test-and-Test-and-Set (TTAS) Lock

```
acquire(L):
        while(L == locked); // wait
        if(test_and_set(L) == locked) go back;
```

  - Idea: Test before you test-and-set
    - Spin on read; no coherence traffic
  - Still a lot of chattering when the lock is unlocked
    - A lot of cache coherence traffic

Better implementations (2)

- Array-Based Lock
  - Idea: Make each thread/process spin on a different memory address
  - Have an array of variables
    - Each variable indicates either lock available or waiting for lock
    - Only one location has lock available

```
next_ticket = 0; // needs to be atomic
flags[0] = AVAILABLE;
flags[1 … N-1] = MUST_WAIT;
acquire(L):
        my_ticket = fetch_and_inc(next_ticket);
        while (flags[my_ticket mod N] == MUST_WAIT);
release(L):
        flags[my_ticket mod N] = MUST_WAIT;
        flags[my_ticket+1 mod N] = AVAILABLE;
```

atomic operation to get value & post increment

Array-based lock example

Suppose three processes, P0, P1, P2.

P0 holds the lock: my_ticket = 0;
flags[0]=AVAILABLE;

P1 and P2 try to acquire the lock:

P1: my_ticket = 1; flags[1] = MUST_WAIT;

P2: my_ticket = 2; flags[2] = MUST_WAIT;

P0 releases the lock;

flags[0] = MUST_WAIT;

flags[1] = AVAILABLE; <- only P0 is writing

P1: can exit from while loop and get the lock

Multikernel Model

The Multikernel: A new OS architecture for scalable multicore systems

Andrew Baumann[*], Paul Barham[†], Pierre-Evariste Dagand[‡], Tim Harris[†], Rebecca Isaacs[†], Simon Peter[*], Timothy Roscoe[*], Adrian Schüpbach[*], and Akhilesh Singhania[*]

[*]*Systems Group, ETH Zurich*

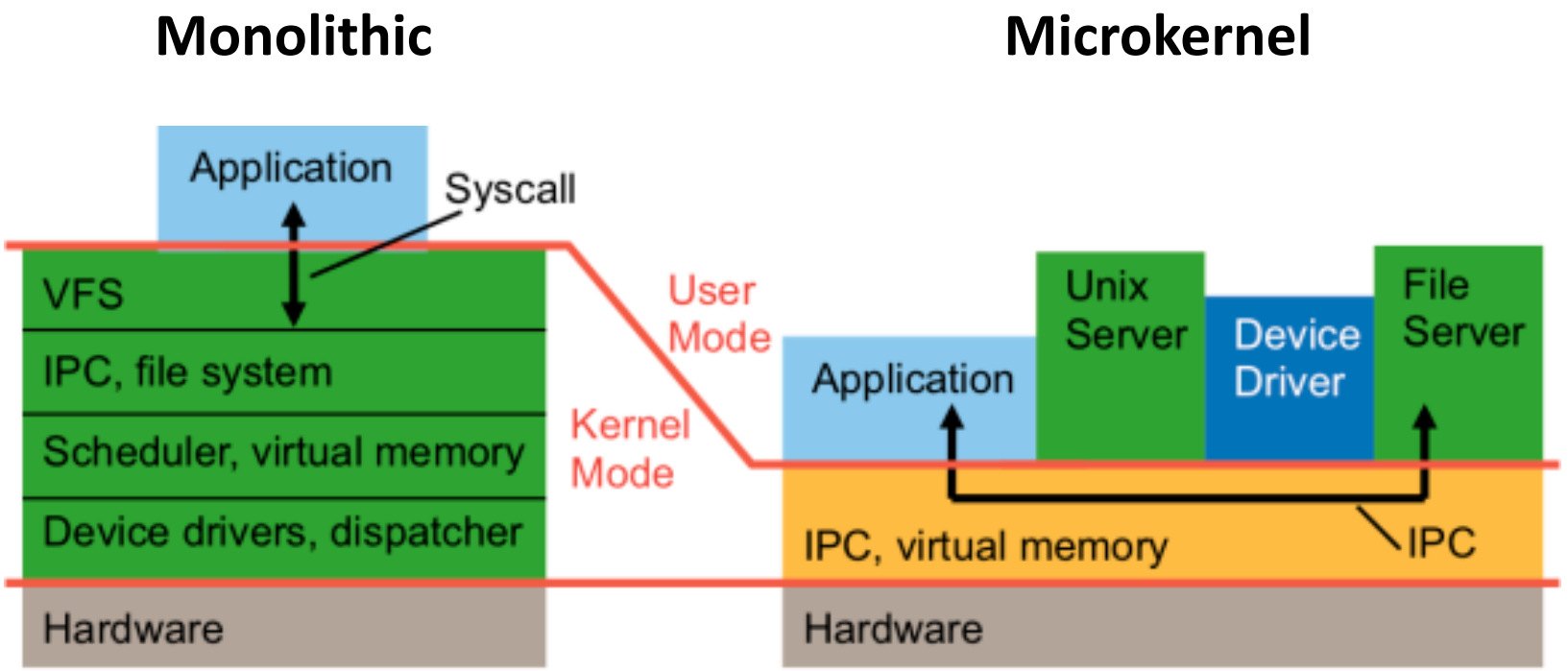
[†]*Microsoft Research, Cambridge* [‡]*ENS Cachan Bretagne*

Key Motivation & Idea

- Scalability & heterogeneity & diversity

- *"We argue that the challenge of future multicore hardware is best met by embracing the networked nature of the machine, rethinking OS architecture using ideas from distributed systems. We investigate a new OS structure, the multikernel, that treats the machine as a network of independent cores, assumes no inter-core sharing at the lowest level, and moves traditional OS functionality to a distributed system of processes that communicate via message-passing"*

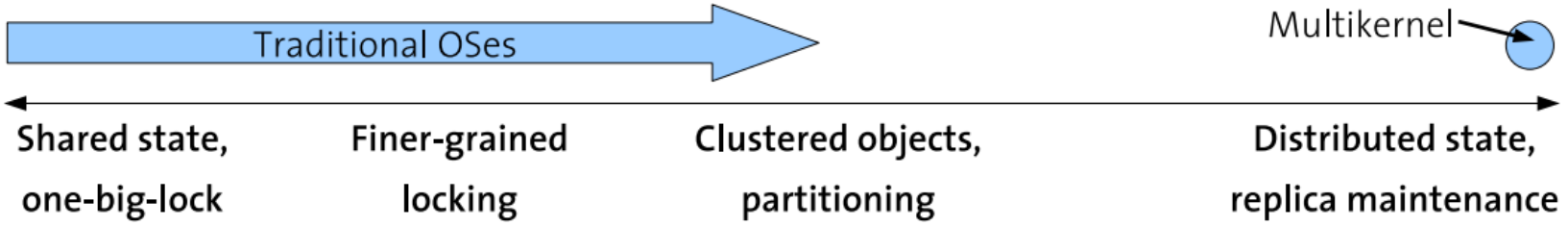
Traditional OSes



Kernel memory space is shared among all CPU cores

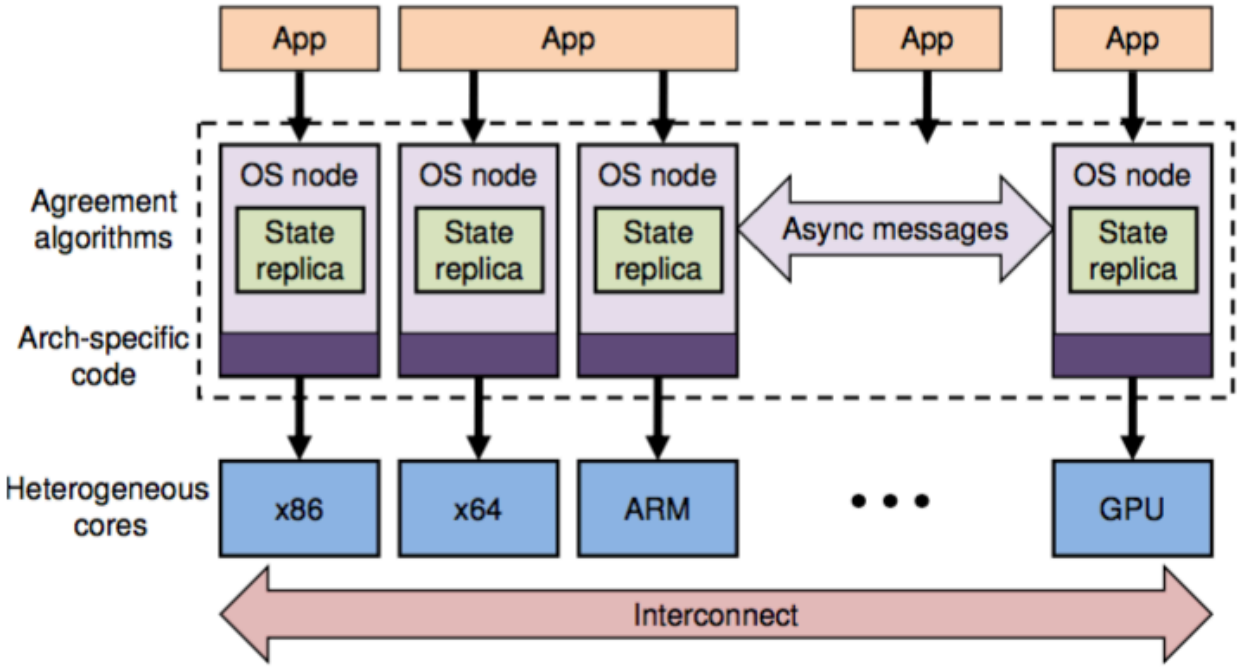
Problems with Traditional OSes

- Scalability issue in the shared memory model
  - Locks!
  - Cache coherence overhead for many cores
- How to scale up traditional kernels?
  - Reducing lock granularity
  - Partitioning states
- Multikernel approach
  - State partitioned/replicated by default rather then shared



MultiKernel model

- **OS as a distributed system of independent cores**
  - Make all inter-core communication explicit (via *message passing*)
  - View state as *replicated* instead of *shared*
  - Make OS structure hardware-neutral



Message passing v.s. shared memory

- Message passing
  - Sending messages between threads or processes, via a message queue or a channel
  - Data exchange or synchronization
  - Dominantly used in distributed systems
- Shared memory
  - Using a region of memory accessible by multiple threads or processes
  - Read/write, synchronization (e.g., using locks)
  - Dominantly used in multiprocessor systems
- Tradeoff space: performance v.s. complexity

Wait! Isn't message passing slower than shared memory?

- Recall microkernel vs. monolithic kernel
- But at scale, message passing ***can*** outperform shared memory
  - Shared memory seems to be plagued by cache misses which cause core stalls

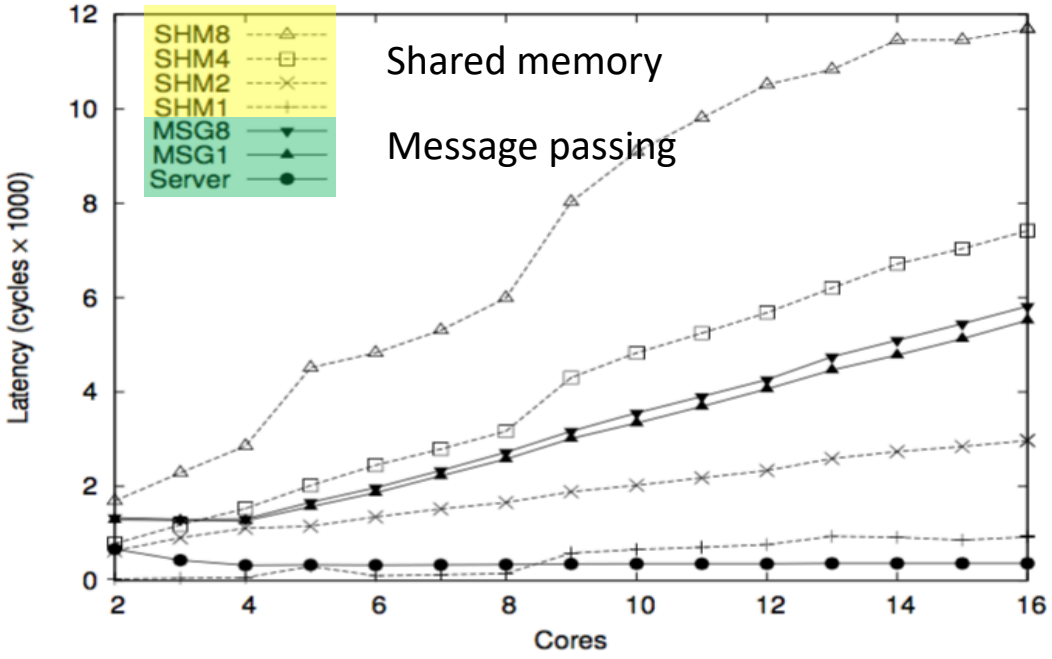


Figure 3: Comparison of the cost of updating shared state using shared memory and message passing.

Make inter-core communication explicit

- No memory is shared between cores
  - Aside from the memory used for messaging channels
- Explicit inter-core communication via messages
  - Provides isolation and resource management on heterogeneous cores
  - Allows the OS to deploy well-known networking optimizations to make more efficient use of the interconnect
- Hardware is starting to resemble a message-passing network

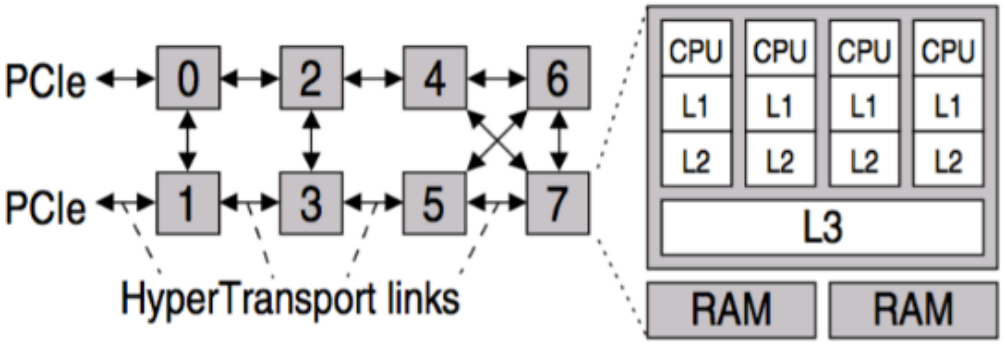


Figure 2: Node layout of an 8×4-core AMD system

View state as replicated

- OS state is replicated and consistency maintained by exchanging messages
  - e.g., scheduler queues
- Why? To improve scalability
  - By reducing load on system interconnect
  - Contention for memory
  - Overhead for synchronization
- Updates may take long; therefore exposed in API as non-blocking and split-phase (= *asynchronous*)

Recall: Synchronous vs. Asynchronous I/O

- Synchronous I/O
  - In most cases, it refers to *blocking* calls
    - e.g., read() or write() blocks a user process until its completion
  - Execution flow does not change
  - Synchronous but *non-blocking* calls exist
    - e.g., read() with O_NONBLOCK flag

```
main() {
    …
    read(…);
    // IO completed
    printf("%c", buf[0]);
    …
}
```

- Asynchronous I/O
  - Inherently *non-blocking*
  - Execution flow can be interrupted by I/O completion
    - e.g., signal handler

```
main() {
    aio_read(…);
    // continue
immediately
    …
}
completion_handler()
{
    printf("%c", buf[0]);
}
```

Make OS structure hardware-neutral

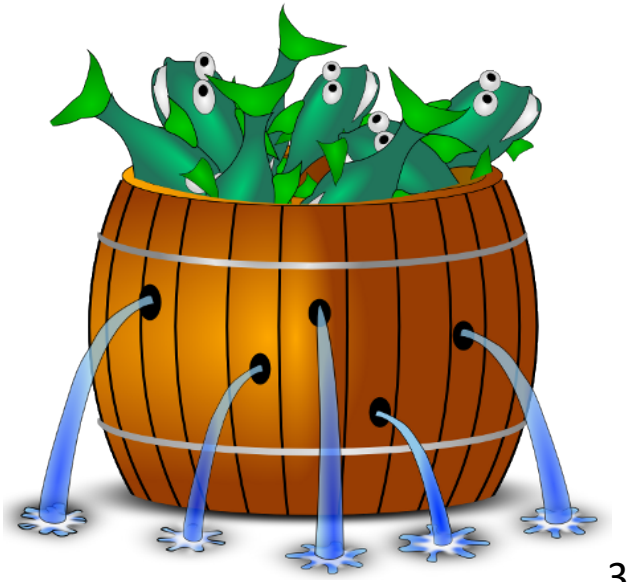
- Separate the OS structure as much as possible from the hardware
  - Why? Adaptation to new hardware features will not require extensive changes to the code base
- Isolate the distributed communication algorithms from hardware implementation details
- Enable late binding of both the protocol implementation & message transport

In practice

- Multikernel model gives an idea but it may not be fully realized
- Idealist message-passing model may sacrifice certain platform-specific performance optimizations – shared L2 cache
- Cost and penalty of ensuring replica consistency varies on workload, data volumes and consistency model
- There can be different ways of implementation... depending on the degree of faithfulness to the model

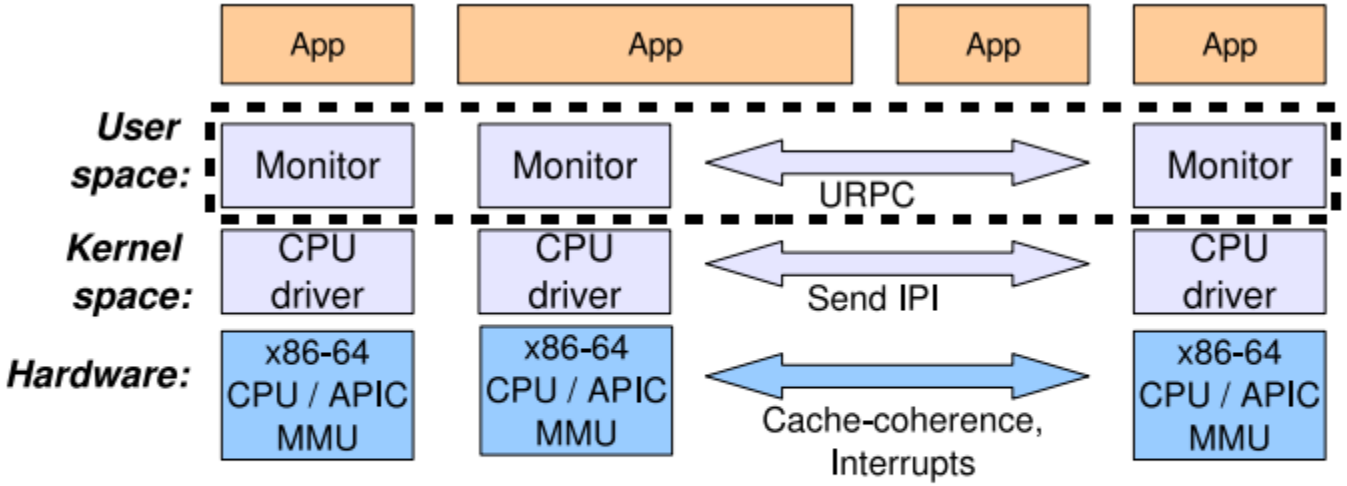
Barrelfish

- Substantial prototype OS following the Multikernel model
- Goals
  - Comparable performance to existing commodity OS on multicore HW
  - Scalability to large number of cores under considerable workload
  - Ability to be re-targeted to different hardware without refactoring
  - Exploit message-passing abstraction for performance
    - pipelining and batching messages
  - Exploit modularity of OS
    - place OS functionality according to hardware topology or load



System Structure

- Multiple independent OS instances communicating via messages
- OS instance on each core
  - Privileged-mode CPU driver (hardware dependent)
  - User-mode monitor process (hardware independent)
    - Responsible for inter-core communication
  - Monitors and CPU drivers provide scheduling, communication and low-level resource allocation



System Components

- CPU Driver (kernel space)
  - Enforces protection, performs authorization, time-slices processes and mediates access to core and hardware
  - Completely event-driven, single-threaded and nonpremptable
  - Minimal functionalities; Serially handles traps and exceptions
    → Microkernel or exokernel-like design

- Monitor
  - Schedulable user-space process → Like services in a microkernel
  - Collectively coordinate consistency of replicated data structures
    - Mediates local operations on global state
    - Replicated data structures are kept globally consistent
  - Wakes up blocked processes in response to messages from other cores

System Components

- Process structure
  - Represented by so-called *dispatchers* objects
  - Communication is between dispatchers
  - Dispatchers on a core are scheduled by the local CPU driver with upcall
  - Dispatcher runs a core-local user-level thread scheduler
    - Similar to *scheduler activations*

- Inter-core communication
  - All communication occurs with messages
  - User-level RPC (URPC): a region of shared memory is used as a channel for cache-line-sized messages
    - No other inter-core messaging mechanism is available on target hardware
  - Implementation is tailored to cache-coherence protocol to minimize interconnect traffic

System Components

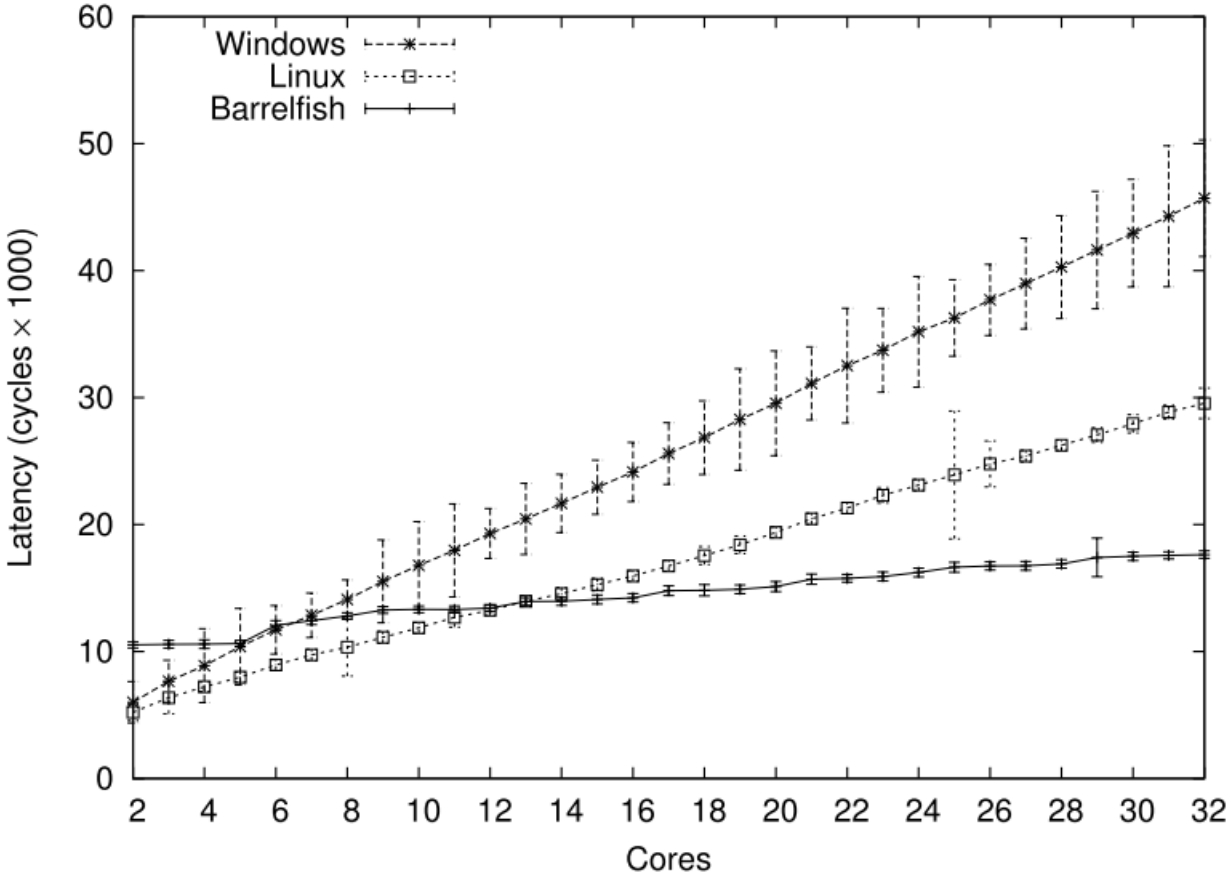
- Memory management
  - Physical memory must be managed as a global resource
    - Because memory can be shared by applications and OS across cores
  - Capability-based memory management
    - Similar to *exokernel*
  - All virtual memory management performed entirely by user-level code
- System knowledge base
  - Maintains knowledge of the underlying hardware
  - Used by OS to derive system policies
    - e.g., allocation of device drivers to cores, NUMA-aware memory allocation in topology aware manner

Experiences from Barrelfish implementation

- Barrelfish is just one concrete implementation of the multi kernel model
- Separation of CPU driver and monitor is not required
  - Not optimal for performance, yet at a constant overhead of local RPC
  - Moving monitor into kernel space is at the cost of complex kernel-mode code base
- Optimization opportunities for the network stack
- Principal difference between existing OS and multikernel
  - Reliance on shared data as default communication mechanism
  - Shared-memory single-kernel model cannot deal with heterogeneous cores

Evaluation

- Unmapping pages



Stable performance from 2 to 32 cores.

Almost 2X Linux at 2 cores, but crossover is around 11 cores

Conclusion

- Scale well with core count
- Feasible alternative to existing monolithic kernels
- Authors say:

> "It would be wrong to draw any quantitative conclusions from our large-scale benchmarks... An enormous investment has been made in optimizing Linux and Windows for current hardware, and conversely our system is inevitably more lightweight (it is new, and less complete). Instead, they should be read as indication that Barrelfish performs reasonably on contemporary hardware..."

Discussion

- Microkernel (and even exokernel) flavor?
  - Isolating hardware specific primitives
  - Customizing implementation
- CPU driver and monitor
  - Nice from software engineering perspective
  - But harmed performance (more border crossings)
- No complex applications were evaluated
  - Only implemented on x86
- Overhead due to consistency & consensus protocols!?
- Hybrid approach?
  - Running each OS instance on a cluster of CPU cores